

Spring 2026 Natural Language Processing (CSCI-6515)

Project 4: Sentiment Analysis using BERT and Reading Comprehension with BiDAF + BERT

Kamal Ahmadov

kahmadov24700@ada.edu.az; kamal.ahmadov@gwu.edu

Rufat Guliyev

rguliyev24988@ada.edu.az; rufat.guliyev@gwu.edu

ADA University & George Washington University

April 8, 2026

Outline

- 1 Project Scope
- 2 Task 1: BERT Sentiment Analysis
- 3 Task 2: Reading Comprehension
- 4 Extra Task: UI
- 5 Conclusion

Motivation and Scope

- Project 4 combines two transformer-related tasks:
 - **Task 1:** theoretical analysis of a fine-tuned BERT sentiment model
 - **Task 2:** implemented QA system with BiDAF and frozen BERT
- The shared question is the same:
 - how useful are contextual transformer representations for downstream NLP tasks?
- Deliverables in this repository:
 - Task 1 report section
 - Task 2 PyTorch implementation + artifacts
 - Streamlit UI for results

Task 1: Selected Open-Source Model

Model analyzed

- `nlptown/bert-base-multilingual-uncased-sentiment`
- Base encoder: `bert-base-multilingual-uncased`
- Fine-tuned for 5-class product-review sentiment

Why this model?

- Open-source and well documented
- More relevant to multilingual transfer than an English-only sentiment model
- Better fit for the Azerbaijani applicability question

Fine-tuning languages

- English, Dutch, German, French, Spanish, Italian

Task 1: Inputs, Outputs, and Light Math

Input pipeline

- Review text $\rightarrow$ WordPiece tokenizer
- Lowercasing because the model is uncased
- Output tensors: `input_ids`, `attention_mask`, optional `token_type_ids`

Classifier output

$$p(y \mid x) = \text{softmax}(Wh_{[\text{CLS}]} + b)$$

- 5 output classes: 1, 2, 3, 4, 5 stars
- Max input size: 512 subword positions

Case sensitivity

- Not case sensitive in practice
- Useful for noisy reviews, but loses emphasis from all-caps text

Task 1: Can It Work for Azerbaijani?

Short answer: yes as a baseline, no as a final solution

Why transfer is possible

- multilingual BERT base encoder
- subword tokenization reduces OOV problems
- useful for morphologically rich languages

Why direct zero-shot use is risky

- no Azerbaijani fine-tuning
- trained on six other languages only
- Azerbaijani is agglutinative, so sentiment-bearing forms appear with many suffixes

Takeaway

- Good starting point for transfer learning
- Reliable Azerbaijani sentiment analysis would still require local fine-tuning

Task 2 Goal and Variants

Goal

- Given a question and a context passage, predict the answer span inside the context

Implemented variants

- **BiDAF + GloVe**: traditional embedding baseline
- **BiDAF + frozen BERT**: contextual embeddings from bert-base-uncased

Dataset

- SQuAD 1.1
- Word-level preprocessing with overlapping context windows

Core engineering choice

- Same BiDAF span-prediction stack for both variants
- Only the embedding source changes

Task 2 Architecture (with light math)

BiDAF pipeline

- Embeddings
- 2-layer highway network
- Contextual BiLSTM
- Bidirectional attention flow
- Modeling layers
- Start/end span heads

Prediction rule

$$(\hat{s}, \hat{e}) = \arg \max_{i \leq j} p_{\text{start}}(i) p_{\text{end}}(j)$$

Training loss

$$\mathcal{L} = \text{CE}(p_{\text{start}}, s^*) + \text{CE}(p_{\text{end}}, e^*)$$

BERT integration

- tokenize words into subtokens
- take first-subtoken hidden state for each word
- project BERT vectors into the BiDAF input space

Task 2 Experimental Setup

Reported run: medium CPU-friendly compare

- Train examples: 512
- Validation examples: 128
- Train windows: 688
- Validation windows: 319
- Epochs: 2
- Batch size: 8
- Hidden size: 32
- Context window: 96 words
- Stride: 48 words

Evaluation

$$\text{EM} = 1[\hat{a} = a^*], \quad \text{F1} = \frac{2PR}{P + R}$$

- Best checkpoint chosen by validation F1
- EM and F1 computed with SQuAD-style normalization

Task 2 Results

Variant	Best epoch	EM	F1
BiDAF + GloVe	1	0.0469	0.1354
BiDAF + frozen BERT	1	0.0234	0.1330

Comparison

- Frozen BERT delta EM: -0.0234
- Frozen BERT delta F1: -0.0024

Interpretation

- The QA pipeline works end to end
- Under this medium setting, frozen BERT did *not* beat the GloVe baseline
- This is a useful negative result, not a failed implementation

Task 2 Error Analysis and Engineering Notes

Why scores are still modest

- medium run, not full benchmark training
- only a subset of SQuAD used for the reported comparison
- BERT encoder is frozen, so it cannot adapt to the QA objective
- exact match is very strict

Important debugging issue we found

- some contexts contained standalone Unicode formatting or combining-mark tokens
- the word tokenizer kept them, but BERT dropped them
- this broke word-level BERT pooling on medium runs

Fix

- filter out tokens made only of control/format or combining-mark characters
- after the fix, the medium compare run completes successfully

Extra Task: Streamlit UI

Implemented UI

- `src/project4_dashboard.py`
- launch with `bash scripts/run_project4_ui.sh`

What the UI shows

- Task 1 BERT model summary
- Task 2 metric boards and comparison charts
- prediction inspector for GloVe vs BERT
- artifact explorer and runbook commands

Why it matters

- easier to demonstrate during presentation
- easier to inspect predictions and saved outputs interactively

Conclusion

Task 1

- A multilingual fine-tuned BERT sentiment model has clear inputs, outputs, and limitations
- It is a reasonable transfer baseline for Azerbaijani, but not a complete solution

Task 2

- We implemented BiDAF in PyTorch and integrated frozen BERT embeddings
- We trained and evaluated on SQuAD with EM and F1
- In the reported medium run, GloVe slightly outperformed frozen BERT

Overall

- All major Project 4 deliverables are present in the repo:
 - report
 - code and experiment artifacts
 - Streamlit UI
 - presentation slides

Thank you

Questions and discussion